

Ön gereksinimler

Bu derste ilk haftadan itibaren C/C++ programlarını **kendi bilgisayarınızda derleyip çalıştırıyor**, projeyi **Git ve GitHub** üzerinde yürütüyor, güvenlik önlemlerini **birim testleriyle** kanıtlıyorsunuz. Bu yüzden aşağıdaki bilgi ve araçlarla gelmeniz gerekir. Dersin resmî ön koşulu **CEN107 Algoritmalar ve Programlama** I'dir; özellikle o dersin ilk haftalarındaki geliştirme ortamı, Git, birim test ve şablon kullanımı bu derste **bilindiği varsayılarak** kullanılır.



İlk dersten önce

Aşağıdaki **5. bölümdeki** denetimleri kendi bilgisayarınızda yapın. Bir adımda takılırsanız ilgili Algoritmalar ve Programlama I sayfasına dönün; hâlâ çözemezseniz ilk derste sorun.

1. Algoritmalar ve Programlama I'den gelenler (zorunlu)

Konu	Nerede öğretiliyor?	Bu derste nerede kullanılıyor?
Geliştirme ortamı: derleyici (GCC/Clang/MSVC), IDE, Windows'ta WSL, CMake ile derleme	CEN107 (CE103) Hafta 2 – Geliştirme ortamları	Her haftanın demoları; 1. hafta laboratuvar kurulumu
Git ve GitHub: depo oluşturma, <code>clone</code> , <code>commit</code> , <code>dal</code> (branch), <code>pull request</code> , <code>.gitignore</code>	CEN107 (CE103) Hafta 3 – Git ile sürüm yönetimi	Proje rehberi (plan, kurulum, teslim); 1. hafta değişiklik yönetimi; 12. hafta sürüm kimliği
Birim test ve kapsam (coverage) araçları: test yazma, <code>ctest</code> ile çalıştırma, kapsam raporu	CEN107 (CE103) Hafta 4 – Birim test ve kütüphaneler	Projede S16 test sonuçları; 4. hafta sanitizer ve fuzzing; 12. hafta test planı
Proje şablonlarının kullanımı: şablonu çatallamak (fork), derlemek, testleri ve dokümantasyonu üretmek	CEN107 (CE103) Hafta 2–4	Proje rehberi · 2. Proje kurulumu – <code>cpp-cmake-ctest-template</code>

İsteğe bağlı başlangıç: [CEN107 \(CE103\) Hafta 1 – Giriş ve geliştirici yol haritası](#).

2. C programlama temelleri (zorunlu)

Demoların hemen hepsi C ile yazılmıştır ve güvenlik hataları çoğunlukla bu temel yapıların yanlış kullanımından doğar. Şunları rahatça okuyup yazabilmelisiniz:

- işaretçiler (pointer), diziler ve dizeler (`char[]`, sonlandırıcı `\0`),
- dinamik bellek (`malloc`, `calloc`, `free`) ve yapı (`struct`),
- dosya okuma/yazma (`fopen`, `fread`, `fgets`),
- birden çok kaynak dosyalı bir programı derlemek ve komut satırı argümanlarını (`argc`, `argv`) kullanmak.

Bu konular **1. hafta** (bellek, taşma) ve **4. hafta** (C/C++ sağlamlaştırma (hardening)) boyunca yoğun biçimde kullanılır.

3. Derste kısaca hatırlatılan ön bilgiler

Aşağıdakileri bilmiyorsanız sorun değil; ilgili haftanın "**Başlamadan önce**" bölümü kısa bir hatırlatma yapar. Önceden göz atarsanız ders daha rahat geçer.

Ön bilgi	Hangi haftada gerekiyor?
Terminal: PowerShell ve Linux/WSL'de <code>cd</code> , <code>ls</code> , dosya yolları, ortam değişkenleri	1. hafta ve bütün demolar
Binary ve onaltılık sayılar, bayt, XOR	3., 9., 11. haftalar
Temel ağ kavramları: istemci-sunucu, TCP/IP, HTTP	3. ve 10. haftalar
Basit SQL (<code>SELECT ... WHERE</code>), Java ya da Python'da temel sözdizimi	5. hafta

4. Bilgisayarınızda kurulu olması gerekenler

Dizüstü bilgisayar gereklidir. İlk derse şu araçlar kurulu gelin:

- C/C++ derleyicisi: Windows'ta **Visual Studio 2022** (C++ ile masaüstü geliştirme), Linux/WSL'de **GCC** ya da **Clang**
- CMake**, **Git** ve bir **GitHub** hesabı
- Windows kullanıyorsanız **WSL2 + Ubuntu**
- OpenSSL 3**, **SQLite**, **Python 3**, **JDK 21**

Dönem içinde ayrıca GoogleTest, SoftHSM2 ve Tigress kullanılır; bunların kurulumu ilgili haftada anlatılır. Demo kodlarının kurulum kılavuzu: [code/README.md](#).

5. Kendinizi sınavın (ilk dersten önce)

Aşağıdaki komutların her biri bir sürüm numarası yazmalıdır (sayılar farklı olabilir):

```
git --version          # git version 2.x
cmake --version        # cmake version 3.2x ya da üstü
wsl --status           # Varsayılan Dağıtım: Ubuntu

git --version          # git version 2.x
cmake --version        # cmake version 3.2x ya da üstü
gcc --version          # gcc (Ubuntu ...) 11 ya da üstü
openssl version        # OpenSSL 3.x
```

Ardından proje şablonunu kendi hesabınıza çatallayıp (fork) derleyin ve testlerini çalıştırın:

```
git clone https://github.com/<kullanici-adiniz>/cpp-cmake-ctest-template.git
cd cpp-cmake-ctest-template
cmake -S . -B build
cmake --build build
ctest --test-dir build      # beklenen: 100% tests passed
```

Bu adımların hepsi hatasız bitiyorsa derse hazırsınız.